

Advance Backend (Docker, Redis, System Design)

Level 1/ Docker:

Docker application+dependencies ko ek box/container me pack kar deta hai.

docker-image: ik blueprint or template hoti h jis sy multiple container bny ja skty hn.

docker-container: running version of image/object created from docker-image.

Why docker: provide isolated environment so app work same on all systems. Provide easy development, scalable, consistent environments, microservices friendly.

Ctrl + D // exit from docker container

Pre-requirements: install docker-desktop app on your system and always run it first only then docker cmds works.

1. Build Your Own Docker-image

```
# dockerfile for nodejs backend inside backend dir
FROM node
WORKDIR /app

COPY package*.json .
COPY . .

RUN npm install
CMD ["node", "index.js"]
```

Process flow:

1st create Dockerfile → docker build -t image_name path → Docker Image → Docker Container

Docker run -it -e PORT=6000 -p host_port:container_port image_name // for port mapping

Go inside docker container:

.dockerignore // here you write files & folder that docker ignore while coping all

Docker run -it image_name //run container through docker-imge

Docker exec -it container_name cmd i.e docker exec -it my_container bash

ls //show all files & folders

mkdir app

cd app

touch index.js

cat index.js // open & see file content in linux

Push Docker-Image on Docker_Hub

1. Create account & login in local terminal using this cmd. -> `docker login`
2. Build docker-image `docker build -t awais37/test-image`
3. Push on docker hub then its public `docker push awais37/test-image`
4. Run it on another system or your own.

`Docker run -it -e PORT=8000 -p 5000:8000 awais37/test-image`

2. Docker Compose (single file to create & run multiple containers)

Level1/phase2

```
>backend      npm init      npm install express dotenv
              /dockerfile
```

```
>frontend      npm create vite@latest      // project-name: . //dot mean current dir and react - js
              /dockerfile & .dockerignore files
```

Docker-compose.yml // here indentation matter – this single file run multiple containers through single cmd. i.e backend & frontend are two separate dir with their dockerfile and this file build their docker-image and run their containers.

```
services:
  backend:
    build: ./backend # build container using its dockerfile
    # environment:
    #   - PORT=7000
    env_file:
      - ./backend/.env # to give env variable file
    ports:
      - 8000:7000

  frontend:
    build: ./frontend
    ports:
      - 5137:5137

  redis:
    image: redis # build container using dockerHub container-image
    ports:
      - 6379:6379
```

your cmd current dir is -> phase2 and then run below cmd to run all 3 container through single cmd.

`docker compose up --build` // to start all containers that are mentioned in .yml file with n updates.

`ctrl + c` //stop containers

`docker compose down` // to stop & remove all containers.

3. Docker Networking

Docker Network allows containers to communicate with each other.

docker network ls // to see all docker network. Bridge is default network of new container.

Docker network inspect network_name // to see how many containers runs on this network.

There total 3 networks: bridge, host, none

- ⇒ Bridge network is default network for new container
- ⇒ Assign specific network to container /or set host network for container so no port mapping needed & we can access container through `http://localhost:port`.
Docker run -it --network network_name image_name
i.e `docker run -it --network host ubuntu` // now container run on host machine network.
Note: if container runs on host network then no need for port mapping.
- ⇒ Network none: mean container run but even with port mapping we not connect with network from local network.

Create Custom Networks:

Docker network create network_name i.e *docker network create awi-net*

why:> if both containers are runs on same network, they can communicate with each other without port mapping.

Steps:

docker login

1. 1st container=> `Docker run -it -e PORT=3000 -p 5000:3000 --network awi-net awais37/test-image`
2. Run 2nd container: `docker run -it -e --network awi-net ubuntu`
3. Open another terminal & run cmd: `docker exec -it container_name bash`
4. Access another container that is on same network using this bash-cmd:
`curl http://another\_cantainer\_name:5000`

4. Docker Volume

Docker volume store data permanently outside the containers, so data remains safe even if containers are deleted or restarted.

Type	Syntax	Data Kahan Save Hota Hai	Use Case	Example Command
Bind Mount	<code>-v "local-folder:container-folder"</code>	Local Machine	Development	<code>docker run -it -v "D:\project:/app" ubuntu</code>
Named Volume	<code>-v volume-name:container-folder</code>	Docker Internal Storage	Database Storage	<code>docker run -it -v mongo-data:/data/db mongo</code>

⇒ **Bind Mount:** (it links container folder with local harddrive folder and ensure both folder have same consistent data, if new data added in container folder then its also added in localdriver folder). **So data stored in local machine harddrive.**

```
docker run -it -v "D:\project:/app" ubuntu
```

// note mentioned path have already dir 'project' and new data & files that we create inside container also stored in that folder. Of our hard drive.

⇒ **Name Volume:** `docker run -it -v mongo-data:/data/db mongo`

// here data saved inside desktop docker – volume place – according to volumne-name

Note: now if we create new data (files & folder) inside container it also stored in volume and we can access it anytime even container was deleted.

➔ how to create new data inside container.

1. Run container in 1 terminal: `docker run -it image_name`
2. Access bash of terminal in 2nd terminal: `docker exec -it container_name bash`
3. Then create files & folders: `cd container_folder` `mkdir myfolder` `touch myfile.txt`

IMPORTANT TASK PRACTICE FOR DOCKER:

1. Apko dockerfile create krni ani chaheya
2. Apko docker-compose file banani ani chaheya.

===== Docker Completed =====

===== Level2/ Redis Start =====

Redis is in-memory key-value database used for super fast data access.

Redis use main memory (RAM) to store data & so server Fastly access this data.

Installation and setup:

>level2

1. Install redis in container by this cmd (its easy installation): `docker run -it -p 6379:6379 redis`
2. Setup backend server: `cd phase1` `npm init`
`npm install express dotenv mongoose`
3. Add "dev" cmd in script
"scripts": {
 "dev": "nodemon index.js"
}
4. `Index.js` `.env` `mongoAtlas_url`

```
import express from "express";
import dotenv from "dotenv";
import connectDB from "../lib/db.js";
dotenv.config();

const port = process.env.PORT || 5000;
const app = express();

app.get("/", (req, res) => {
  res.status(200).json({ message: "Wellcome to our server" });
});

app.listen(port, () => {
  connectDB();
  console.log(`server is listening on port ${port}`);
});
```

5. signIn & create cluster – create user – mongodb & vscode – copy dbURL & paste into .env file
`mongodb+srv://joyiashab267_db_user:eMwieWdaj1S5uKyW@cluster0.tavmqvg.mongodb.net`
remove '/' at end
6. Database and Network Access -> IP access list -> delete old ip & add 0.0.0.0/0
7. Level2/phase1/model/user.model.js

```
import mongoose from "mongoose";

const userSchema = await mongoose.Schema(  

```

```

{
  name: String,
  email: String,
  password: String,
},
{ timestamps: true },
);

const User = mongoose.model("User", userSchmea);
export default User;

```

8. Lib/db.js

```

import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URL);
    console.log("db connected");
  } catch (error) {
    console.log(error);
  }
};

export default connectDB;

```

1. Redis API chaching / (store mostly use data of api in redis for fast access)

1. Install ioredis package in server dir: npm install ioredis
2. Add env variable in .env: REDIS_URL=redis://localhost:6379 //note redis container run in docker
3. Inside index.js const redis = new Redis(process.env.REDIS_URL)
4. Request with redis:

```

app.get("/get-with-redis", async (req, res) => {
  const chached = await redis.get("user:all"); //key
  if (chached) {
    const user = JSON.parse(chached); // string to json data converting
    return res.json(user); //if data found in redis then return it.
  }

  const user = await User.find({});
  await redis.set("user:all", JSON.stringify(user)); //json to string convert
  // through .stringify, to store string (key-value) data in redis
  res.status(200).json(user);
});

```

// note: redis store data in string key-value format

Note: redis is used to cache data, here you compare normal get request vs get-with-redis and see how redis provide fast data access on /get-with-redis.

5. Update redis data:

we delete old redis data on given redis-key while creating new user, so after that if we get user then its show all new update data/user on given key.

```
app.post("/create", async (req, res) => {
  const { name, email, password } = req.body;
  await redis.del("user:all"); // delete old redis data so it get latest data
  // after new user created
  const user = await User.create({ name, email, password });
  res.status(200).json(user);
});
```

Learnings:

```
Const chached = Redis.get("cache_key") // get data in redis
JSON.parse(chached) //parse data string into json
Redis.set("cache_key", "value") // set data in redis in string format
JSON.stringify(chached) //convert json into string while seting data.
Redis.del("cache_key") // del data in redis
```

2. OTP storage / redis have built-in option for otp expire:

We temporary store OTP in redis for fast access and verification. set otp in 'send-otp' rout

```
await redis.set(`otp:${email}`, otp, "EX", 60); // key, value, option: expire
in 30sec
```

```
const chached = await redis.get(`otp:${email}`);
```

level2/phase1/index.js //get otp example in 'verif-otp' route

```
app.post("/verify-otp", async (req, res) => {
  const { email, otp } = req.body;

  const chached = await redis.get(`otp:${email}`);
  if (!chached) return res.json({ message: "opt not found or expired" });
  if (chached !== otp) return res.json("your otp is not correct");

  res.status(200).json({ message: "otp is successfully verified" });
});
```

3. Session Store in Redis / i.e jwt token store in redis.

The slide features the Redis logo at the top left. Below it, the text 'Sessions:' is followed by a large heading 'Login Flow'. To the left of the code snippets is a vertical flowchart with four steps: 'Login', 'Generate JWT', 'Store Session In Redis', and 'Cookie Send', connected by downward arrows. To the right of the flowchart are two code snippets. The first snippet shows the creation of a JWT token using the 'jsonwebtoken' library. The second snippet shows the storage of the token in Redis with an expiration time of 86400 seconds, followed by setting the token as a cookie on the response.

```
const token = jwt.sign(
  { userId: user.id },
  secret
)

await redis.set(
  `session:${user.id}`,
  token,
  "EX",
  86400
)

res.cookie("token", token)
```

4. Rate Limit using Redis (one of important usecase of redis)

Rate-limiting means 'Ek time ma limited request allow kro specific api ko'.

it helps to prevent these attacks: that cause slow & crash server.

Brute Force (password guess), Spam(spam api), DDos(server overload), Bot Abuse(fake traffic)

Redis rate limit technique:

we count each request in redis in 1mint range. If limit cross server block that api & unblock after 1mint.

request → count request → Limit Cross? → Block for 1mint

Redis use these cmds:

INCR // count increase

EXPIRE // reset count

Level2/phase1/middleware/rateLimiter.js

```
import { redis } from "../index.js"; #export const redis = new redis(redis url) in index.js

//create rateLimiter middleware
const rateLimiter = async (req, res, next) => {
  const ip = req.ip;
  const key = `rate-limit:${ip}`;
  const req_count = await redis.incr(key);

  if (req_count == 1) await redis.expire(key, 60); //key, exp-time after 60sec
  if (req_count > 5) {
    return res.status(429).json({ message: "Too many requests" });
  }

  next();
};

export default rateLimiter;
```

Level2/phase1/index.js

```
app.get("/get", rateLimiter, async (req, res) => {
  const user = await User.find({}); #find all users
  res.status(200).json(user);
});
```

Note: now if user send more then 5 request with in 1mint, user-ip blocked for 1mint duration. After a minit it auto unblock.

rate-limiting ensure only limited(i.e 5 or 100 in real world system) requests allowed per mint.

RateLimit api hum hr jagha nhe lgaty... commonly esy search, get, login (sb sy main h) is tarha ki requests k sth ratelimiting lgaty hn. So koi bar bar login kr password guess na kry sky or ddos attack bhe na ho.

5. Queues in Redis

Queue is data structure that follow FIFO order.

Queue ek waiting line hoti hai jisme backend tasks store hote hain taaki unhe background me process kiya ja sake bina user ko wait karaye.

Worker: background ma Queue k task perform krny wla.

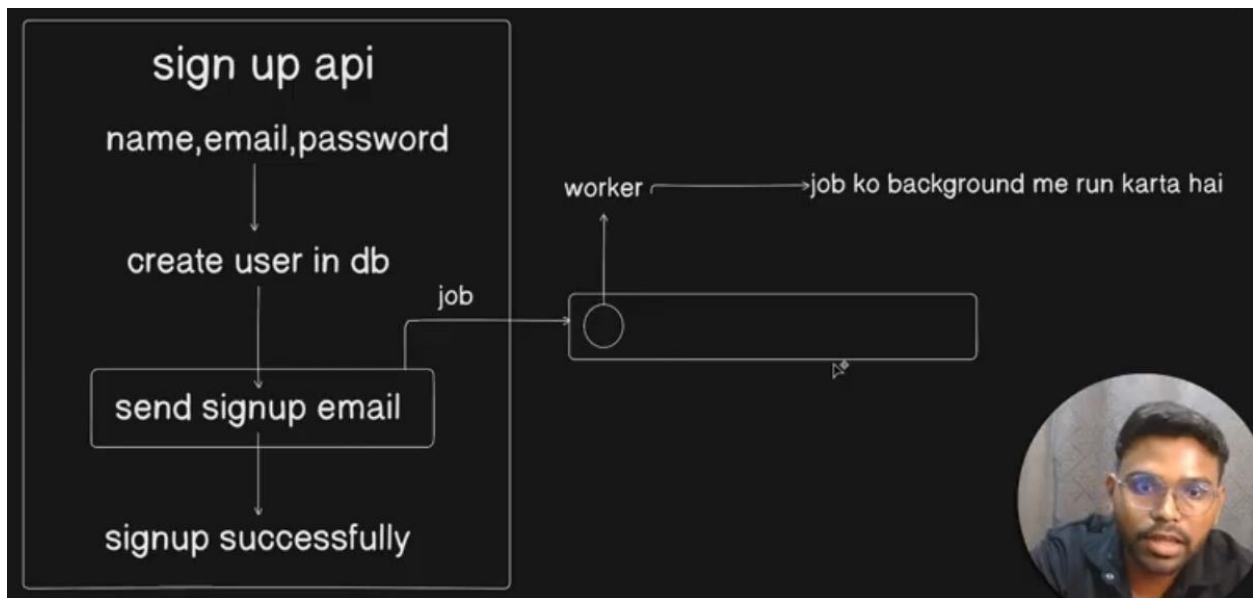
Example: user account create kry – signup email snd ho – then dashboard access milay.

Yha user ko email snd hony ka wait krna pry ga jis sy user experience krab hota h. so 'send signup email' task ko hm queue ma rakhwa dein gy background processing k liay. And user ko instant dashboard access dy dein gy... to improve user experience. Different tasks k liay different queues hoti hn.

BullMQ

one of most popular redis-based queue management library, that handle background jobs of queue.

queue → job store, job means task, worker → process jobs/tasks



i.e you can implement queue for specific task 'send-signup-email'. Push task in queue for background processing.

But for now I implement same queue in 'send-otp' route & for 'send-otp-mail' task that push to queue and worker process it.

Npm install bullmq

Level2/phase1/queue.js

```
import { Queue } from "bullmq";
import Redis from "ioredis";

//create connection
const connection = new Redis("redis://localhost:6379", {
  maxRetriesPerRequest: null, #unlimited try to connect to server it failed to connect
});
```

```
// create queue
const emailQueue = new Queue("emailQueue", { connection }); #pass conn as {obj}

export default emailQueue;
```

level2/phase1/index.js // for example add task 'send-email' in queue.

```
app.post("/send-otp", async (req, res) => {
  const { email } = req.body;
  const otp = Math.floor(100000 + Math.random() * 900000).toString();

  //add task in queue for background processing
  await emailQueue.add("send-email", { transporter, otp, email });

  await redis.set(`otp:${email}`, otp, "EX", 60); // key, value, option: expire
  // in 30sec
  console.log("otp is: ", otp);
  res.status(200).json({ message: "OTP sent to your email", result });
});
```

Level2/Phase1/Worker.js

```
import { Queue, Worker } from "bullmq";
import Redis from "ioredis";
import { sendMail } from "../lib/sendMail.js";
import { transporter } from "../index.js";

//create Redis Connection
const connection = new Redis("redis://localhost:6379", {
  maxRetriesPerRequest: null,
});

//create worker
const worker = new Worker(
  "emailQueue",
  async (job) => {
    console.log("job started");
    const email = job.data.email;
    const otp = job.data.otp;

    await sendMail(transporter, otp, email);
    console.log("job completed");
  },
  { connection },
);
```

note: you to run worker in seprate teminal and also run server on other terminal

level2/docker-compose.yml //run to create & run redis server

```
services:
  redis:
    image: redis
    ports:
      - 6379:6379
```

Make sure redis container, worker.js and server(index.js) all running to implement queue & workers.

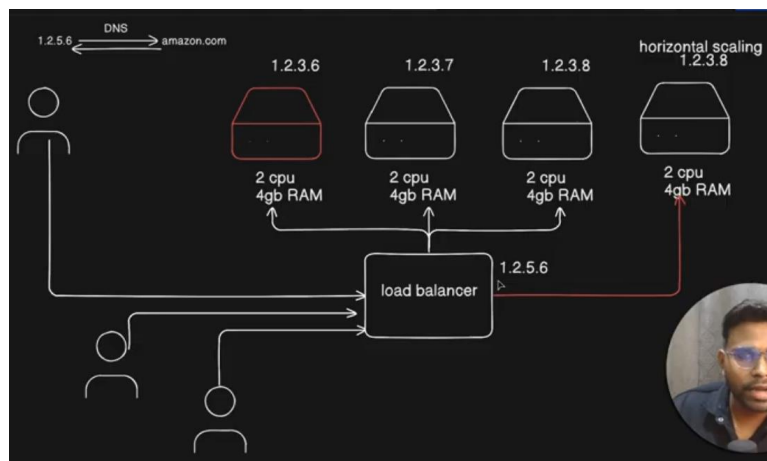
===== Level3/ System Design =====

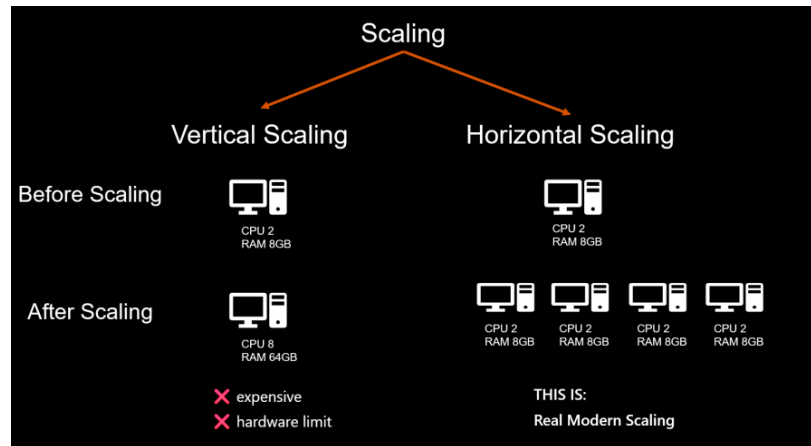
System Design is the process of designing scalable, reliable, fast, and production-ready software architectures.

Technology	Purpose
Node.js	backend
Redis	cache
Docker	containers
Nginx	load balancer
MongoDB	database

In DNS we use 'load balancer' ip address.

Load Balancer use 'round robin' algorithm for load balancing.





Load Balancer:

Type of server (load balancer) that distribute incoming traffic across multiple backend servers. It uses round robin algorithm.

Real load balancer options: type

1. Nginx software (commonly used in websites)
2. AWS ELB cloud service

Nginx:

Nginx is reverse proxy and load balancer used to manage traffic between users and backend servers. jb load aye ga ya kuxh traffic server 1 pr, kuxh server 2 pr so on... pr bhej dy ga. Ta ky single server overload na ho.

User → Nginx → Backend1 or Backend2

Nginx features:

1. **Reverse Proxy** // is a server that sits in front of web servers and intercepts incoming client requests before they reach the backend. User → reverse proxy (intermediate server) → backend servers.
2. **Load balancing** // the practice of distributing incoming network traffic across multiple backend servers
3. **SSL** // SSL (Secure Sockets Layer) is the standard security technology that establishes an encrypted link between a web server and a browser. In modern era TLS is better than SSL.
4. **Routing** // directing user webrequest to correct functional code or view.

Nginx installation:

Install it through docker container.

Level3/docker-compose.yml

```
services:
  nginx:
    image: nginx
    ports:
      - 8000:80
```

Docker compose up //cmd to create & run nginx server container.

Go inside container: `docker exec -it container_name bash` //makesure container already running.
`ls` `cd etc` `ls` `cd nginx` `cat nginx.conf` // file were we config load balancer.

To edit nginx.conf in terminal: we have 2 methods

1. Install vim // it edit files in terminal (optional)
2. Using docker-volume folder mount method. // is sy agr hm docker k folder ko local disk k folder k sth mouth/link kr lety hn. Then agr hm local folder ma change krein gy tu woh docker wli file ma bhe change hoga or agr docker file ma change krien gy tu local file ma bhe ho jay ga because both linked.



Yha hm ny local disk ma 'level3/nginx/nginx.conf' file bnai and osy docker the file sy moute/connect kr dia. Ab hm jo local nginx.conf ma changes krien gy woh direct docker ki file nginx.conf ma reflect hn gi so is sy hm apny load-balancer ko config krein gy.

level3/nginx/nginx.conf

```
# boilercode of nginx
events{

}

http {
    server {
        listen 80;    #server listing port
        location / { # write custom logic for '/' route - load balance
            return 200 "helo from nginx";
        }
    }
}
```

Docker compose down

`docker compose up --build` //docker remount volume also

After that changes & updates occur in real time. No need to compose down & up. Just make changes in nginx.conf file that reflect automatically in real time.

1. Create Multiple server using same server dir & dockerfile. Then load-balancer

Level3/server // inside server dir, copy paste level2-phase1 code for server and add dockerfile

Level3/server/dockerfile

```
FROM node #base-image
WORKDIR /app

COPY package*.json .
COPY . .

RUN npm install #run in runtime to install pkgs
CMD ["node", "index.js"]
```

Level3/docker-compose.yml

```
services:
  redis: #1st container name
    image: redis:7-alpine
    ports:
      - 6379:6379

  nginx:
    image: nginx # here it pull image from docker-hub
    ports:
      - 8000:80
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    depends_on: # means nginx wait for 'depends' then after that nginx start.
      - server1
      - server2
      - server3

  server1:
    build: ./server # here we have custom-dockerfile to create container
    env_file:
      - ./server/.env
    environment: # create env variable in run time
      - SERVER_NAME="pehla server"
      - REDIS_URL=redis://redis:6379
    ports:
      - 5001:7000
    depends_on:
      - redis
```

```

server2:
  build: ./server # here we have custom-dockerfile to create container
  env_file:
    - ./server/.env
  environment:
    - SERVER_NAME="dusra server"
    - REDIS_URL=redis://redis:6379
  ports:
    - 5002:7000
  depends_on:
    - redis

server3:
  build: ./server # here we have custom-dockerfile to create container
  env_file:
    - ./server/.env
  environment:
    - SERVER_NAME="tesra server"
    - REDIS_URL=redis://redis:6379
  ports:
    - 5003:7000
  depends_on:
    - redis

```

Note: here 3 server container run on same 7000 port because each container is isolated machine so where every container have separate all ports like server1 7000 port which is different from server2 container 7000port.

Level3/server/index.js //add SERVER_NAME env that created runtime.

```

app.get("/", async (req, res) => {
  res
    .status(200)
    .json({ message: `Wellcome from MERN ${process.env.SERVER_NAME}` });
});

```

Level3/nginx/nginx.conf

```

# boilercode of nginx
events{

}

```



```

http {
    upstream backend { #create variable 'backend' here we write all server urls
                        except load balancer.
        server server1:7000;
        server server2:7000;
        server server3:7000;
    }

    server {
        listen 80; #server listing port
        # write custom logic for '/' route - load balance
        location / {
            proxy_pass http://backend;
        }
    }
}

```

Note: Load balancer nginx use round robin algorithm, means if we have 3 servers.

1st request goes to → server1 2nd request goes to → server2 3rd request → server3

4th request → again server1 5th req → server2 so on....

Final step build server & send 4 request on 'http://localhost:8000/' to analyse behavior

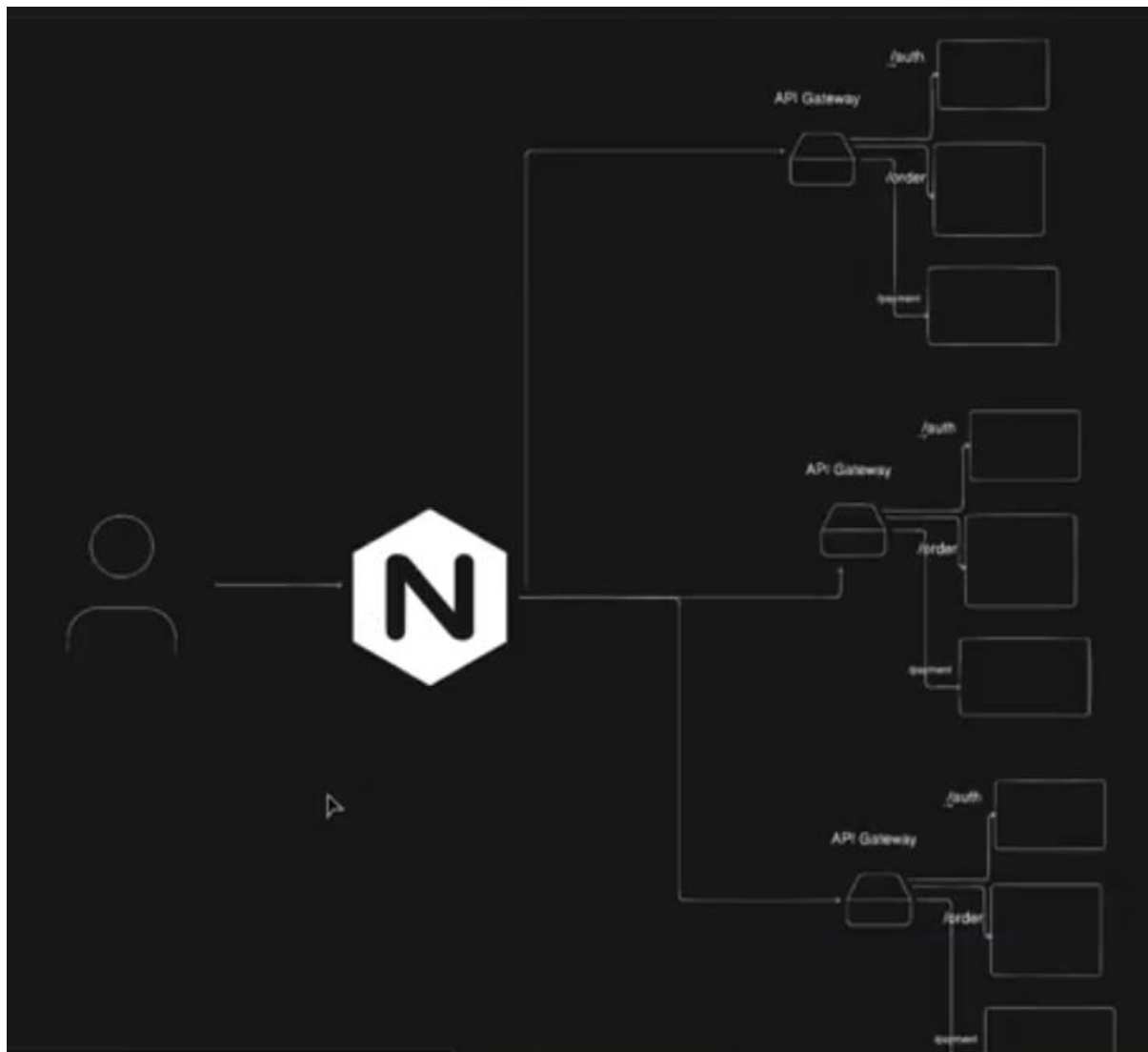
Docker compose down docker compose up --build

Load Check: //process to check scaling, means how many user available servers handle easily

Open cmd & npm install **autocannon**

Autocannon k zrya hm fake request load snd kr skty hn server pr then report generate ho kr aye gi k server kitny load ko handle kr skta h.

Autocannon -c 100 -d 10 <http://localhost:800/> //concurrent requests, duration 10sec req_url



Monolith: jb normaly ik hi server sari services ko perform krta h, tu osy monolith kehty hn.

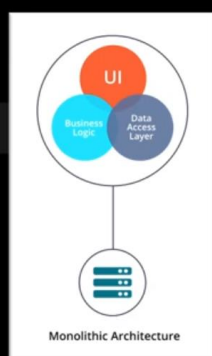
2. Monolith and Microservices :

Monolith

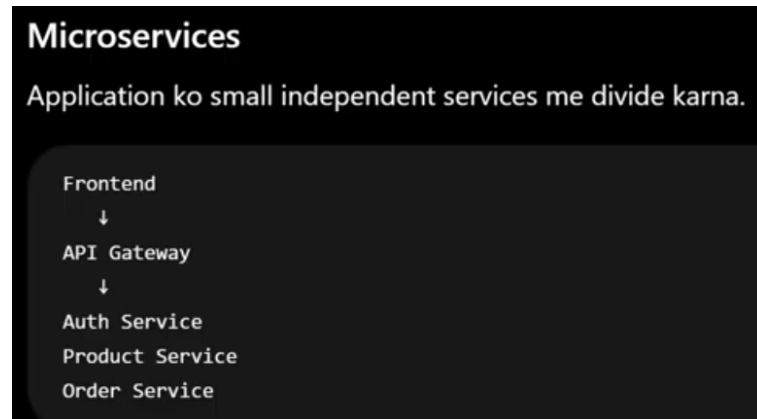
Ek single large application architecture.

Frontend → One Backend → One Database

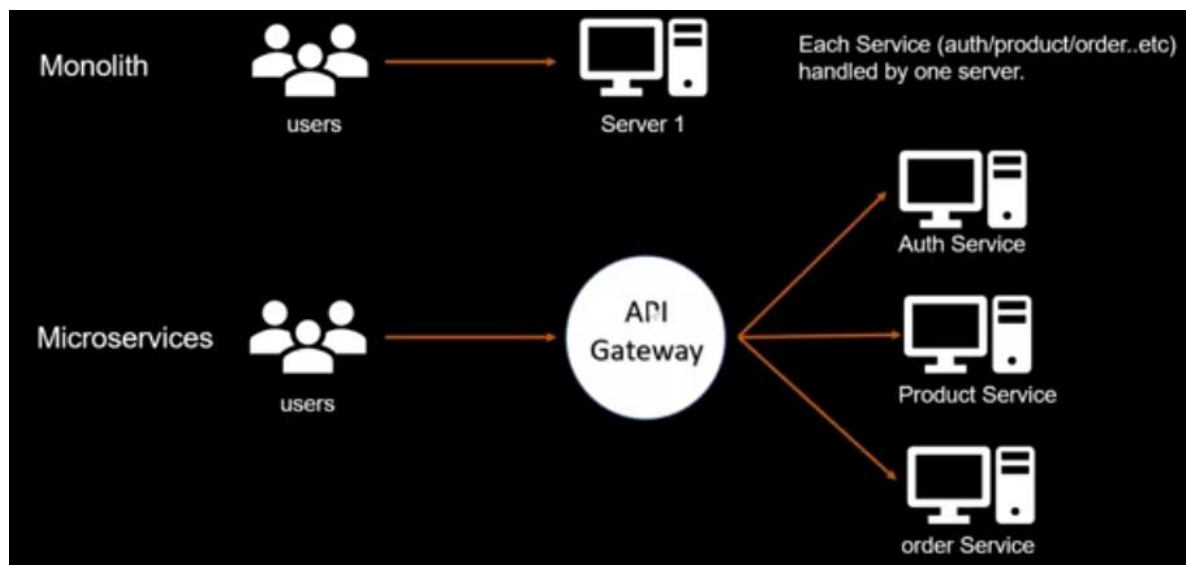
Sab modules ek hi project me.



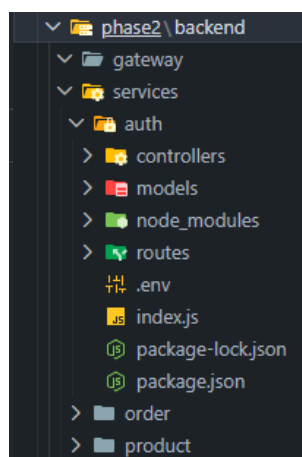
Microservices: hr ik service k liay alag server jb bnaty hn tu esy kehty hn microservices.



Understand them through diagram



Level3/phase2/backend/



Level3/phase2/backend/services/auth/index.js

```
import express from "express";
import dotenv from "dotenv";

dotenv.config();

const port = process.env.PORT || 5000;
const app = express();

app.get("/", (req, res) => {
  res.status(200).json({ message: `wellcome from auth service` });
});

app.listen(port, () => {
  console.log(`Server is Listing on port ${port}`);
});
```

Similary setup order & product services servers like auth and run all services simulationusly through multiple terminal node index.js

Port change for every sercies : 801, 802, 803

Level3/phase2/backend/gateway

Npm init -y npm i express dotenv **express-http-proxy** //express-http-proxy pkg for gateway

Gateway.index.js

```
import express from "express";
import dotenv from "dotenv";
import proxy from "express-http-proxy";

dotenv.config();

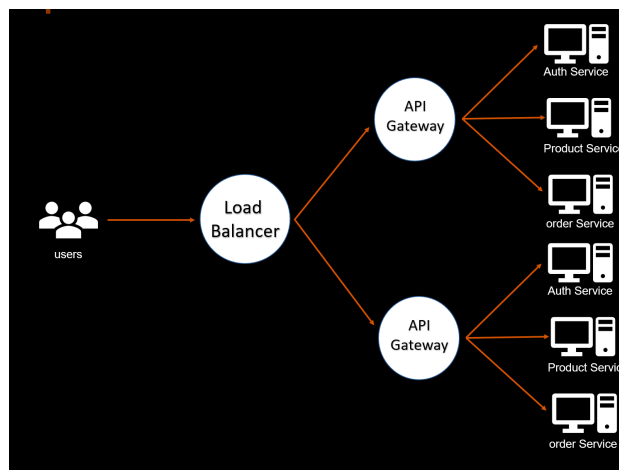
const port = process.env.PORT || 8000;
const app = express();

////////////////////////////////////
//http://localhost:8000
app.use("/auth", proxy("http://localhost:8001"));
app.use("/order", proxy("http://localhost:8002"));
app.use("/product", proxy("http://localhost:8003"));
////////////////////////////////////
```

```
app.get("/", (req, res) => {
  res.status(200).json({ message: `wellcome to main server or gateway` });
});

app.listen(port, () => {
  console.log(`Server is Listing on port ${port}`);
});
```

3. Horizontal Scaling in Microservices (create multiple api gateways managed by load balancer, services remains same)



Process is simple we create dockerfile for every server dir to build docker-image & containers through that image. User docker to run multiple api gateways and services, one load balance using nginx image. through single api-gateway docker-image we build multiple containers and enable horizontal scaling.

Steps:

1. Add dockerfile in every server dir to build its docker-image & containers (i.e for service server, gateway server, etc)

Dockerfile // same dockerfile for gateway, (auth, order, product)services. Each have one dockerfile

```
FROM node
WORKDIR /app

COPY package*.json .
COPY . .

RUN "npm install"
CMD ["node", "index.js"]
```

2. Create docker-compose.yml in level3/phase2/backend/docker-compose.yml
// it creates all servers (gateway, services, load-balancer through single file)

```
services:
  nginx: # container name
    image: nginx # pull image from dockerHub
    ports:
      - 8000:80
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf #moute container file with local
one, so we make config in nginx server directly throguh vscode
    depends_on: #startup timing, frist below containers run then nginx run
      - gateway1
      - gateway2

  gateway1:
    build: ./gateway
    env_file: ./gateway/.env
    ports:
      - 7001:8000
    environment:
      - SERVER_NAME=pehla gateway

  gateway2:
    build: ./gateway
    env_file: ./gateway/.env
    environment:
      - SERVER_NAME=dusra gateway
    ports:
      - 7002:8000

  auth-service:
    build: ./services/auth # here you found dockerfile to build image&container
    env_file: ./services/auth/.env
    ports:
      - 5001:8001

  order-service:
    build: ./services/order
    env_file: ./services/order/.env
    ports:
      - 5002:8002

  product-service:
    build: ./services/product
```

```
env_file: ./services/product/.env
ports:
  - 5003:8003
```

// if you see docker-compose.yml you notice for now we created two gates to enable horizontal microservices scaling. You can create more as much needed for specific project.

3. load-balancer setup:

create folder & file to config nginx like level3/phase2/backend/nginx/nginx.conf

```
events {
}

http {
    upstream backend {          #creating variable 'backend'
        server gateway1:8000;   #container url:port - not local url:port
        server gateway2:8000;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://backend; #here backend is variable name
        }
    }
}

# for now nginx have 2 gateways to balance load
# nginx server balance load using round robin algorithm, send 1st request on
gateway 1, 2nd request on gateway2, 3rd request on gateway1 again.
```

4. Gateway route url changed (local to containerize)

<http://localhost:8001> to <http://auth-service/8001>

```
import express from "express";
import dotenv from "dotenv";
import proxy from "express-http-proxy";

dotenv.config();

const port = process.env.PORT || 8000;
const app = express();

//http://localhost:8000 change localhost to containerize
app.use("/auth", proxy("http://auth-service:8001"));
```



```

app.use("/order", proxy("http://order-service:8002"));
app.use("/product", proxy("http://product-service:8003"));

app.get("/", (req, res) => {
  res.status(200).json({ message: `wellcome to main server or gateway` });
});

app.listen(port, () => {
  console.log(`Server is Listing on port ${port}`);
});

```

Similary update url in order & product service 'index.js'

Run below cmds to build docker images & containers

Your current dir: level3/phase2/backend/

Docker compose up

Docker compose down //if error occurs delet old composed images & containers

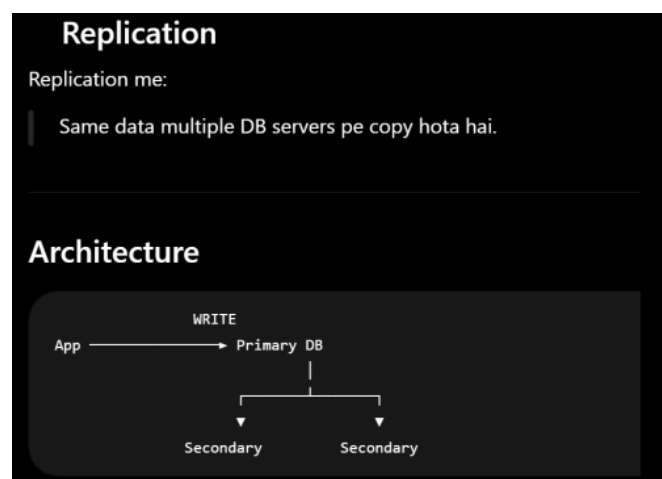
Docker compose up --build // rebuild to add apply new changes

4. Database Scaling

Database ko itna powerful banana ki wo ziyda users, ziyada requests aur ziyada data ko handle kr sake.

Horizontal data scaling 2 main concepts:

1. Replication (one primary db for write opr and other secondary db(that are copy of primary db for read operations), because 90% read operations 10% write operations)
2. Sharding



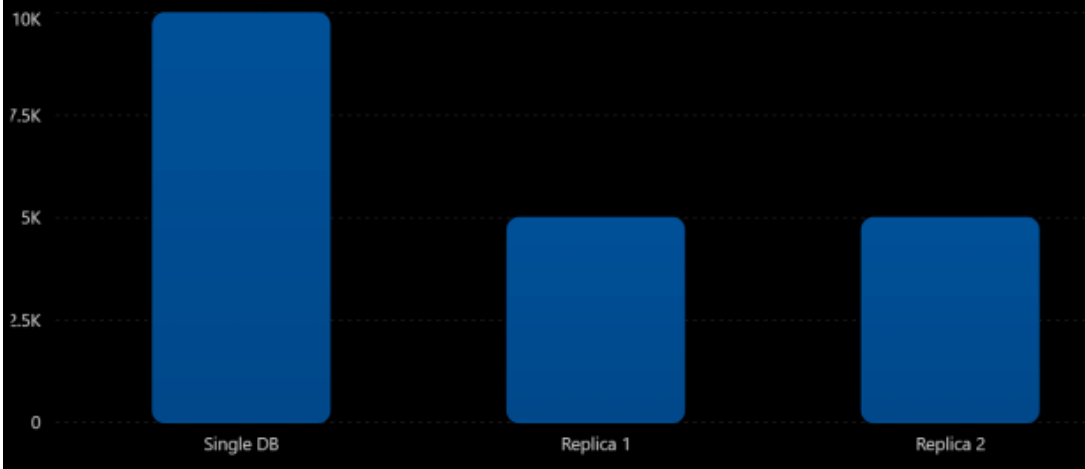
Primary DB server use to write data.

Secondary DB server user for Read + Backup.

Read Scaling

Read load distribution with replication

Replication distributes read traffic across multiple replica nodes.



Sharding

Sharding Meaning

Data ko divide karna multiple databases me.

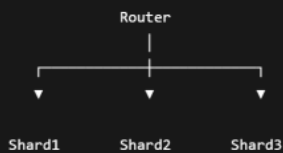
If company have 1m users then it use sharding. Because now single db not handle huge data.

Example

Users split:

Users 1-1M	→ Shard 1
Users 1M-2M	→ Shard 2
Users 2M-3M	→ Shard 3

Diagram



Sharding Solves

- ✓ write scaling
- ✓ huge data
- ✓ storage scaling

Replication vs Sharding

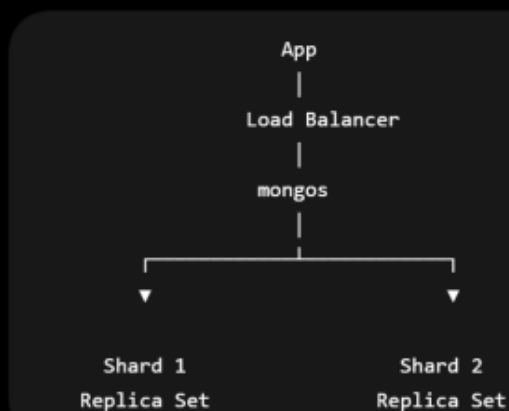
Feature	Replication	Sharding
Purpose	Read scaling	Write/data scaling
Data	Same copy	Divided
Storage	Repeated	Distributed
Complexity	Easier	Harder

Replication increases availability and read performance.
Sharding increases storage and write scalability.

Real Production Setup

Companies dono combine karti hain.

Architecture



MongoDB automatically manage replication in free plan and sharding in pro plan.

Every website has different system design i.e youtube & jio star have different system design. So previous traffic & future traffic prediction k sehb sy hm system design krty hn.